

OBJECT-ORIENTED DESIGN · C++

Visitor

```
accept(visitor);
```

BEHAVIORAL
PATTERN

Group 03 - CS202

Table of Contents

01

INTRODUCTION

02

BLUEPRINT

03

SAMPLE CODE

04

CONSEQUENCES & APPLICATION

05

COMPARISON & RELATION

06

QUIZ

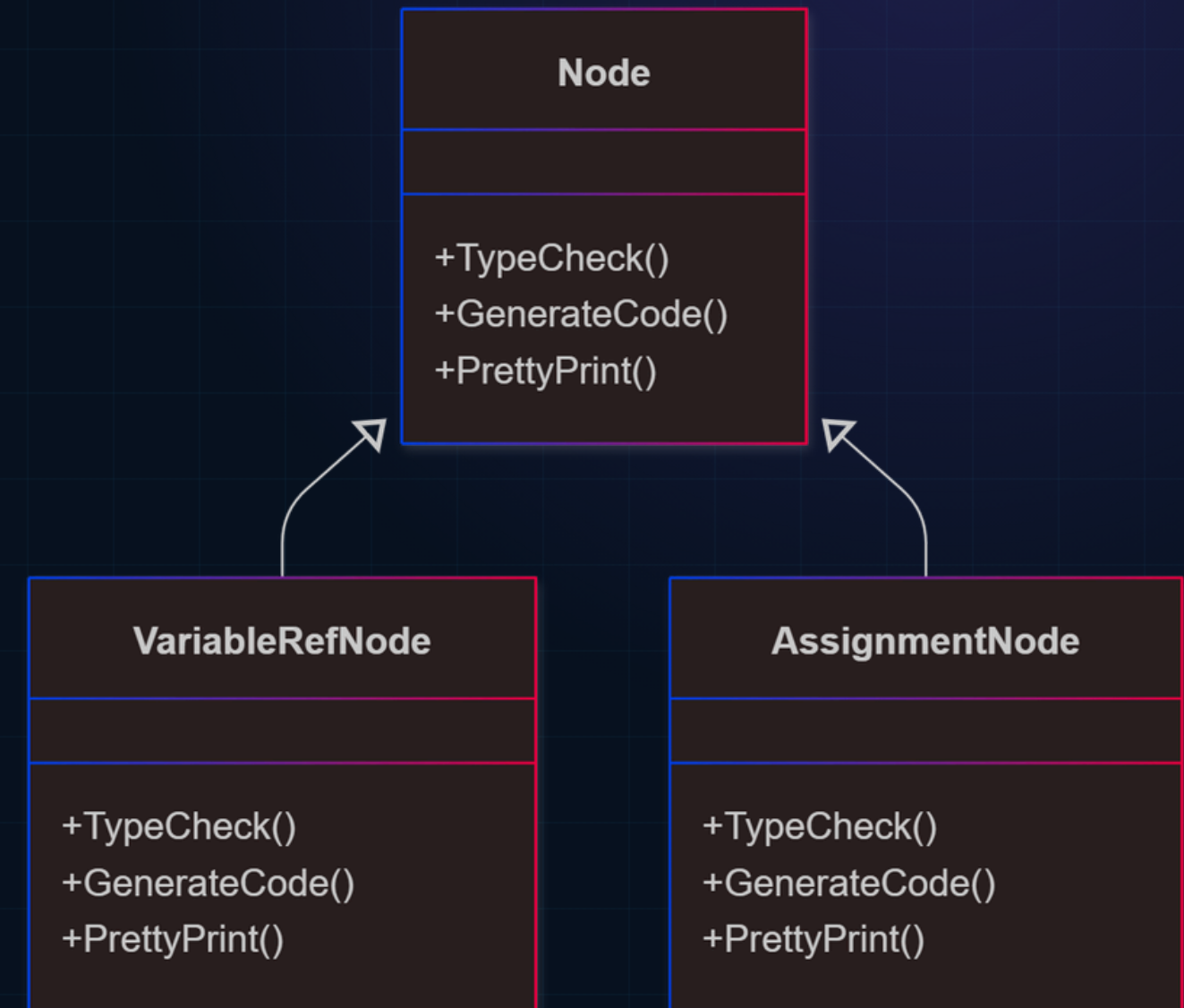
Start With the Compiler Problem

- 01 Imagine you are building a compiler that represents programs as abstract syntax trees (ASTs).
- 02 It will need to perform operations on abstract syntax trees for "static semantic" analyses.

Naive Solution

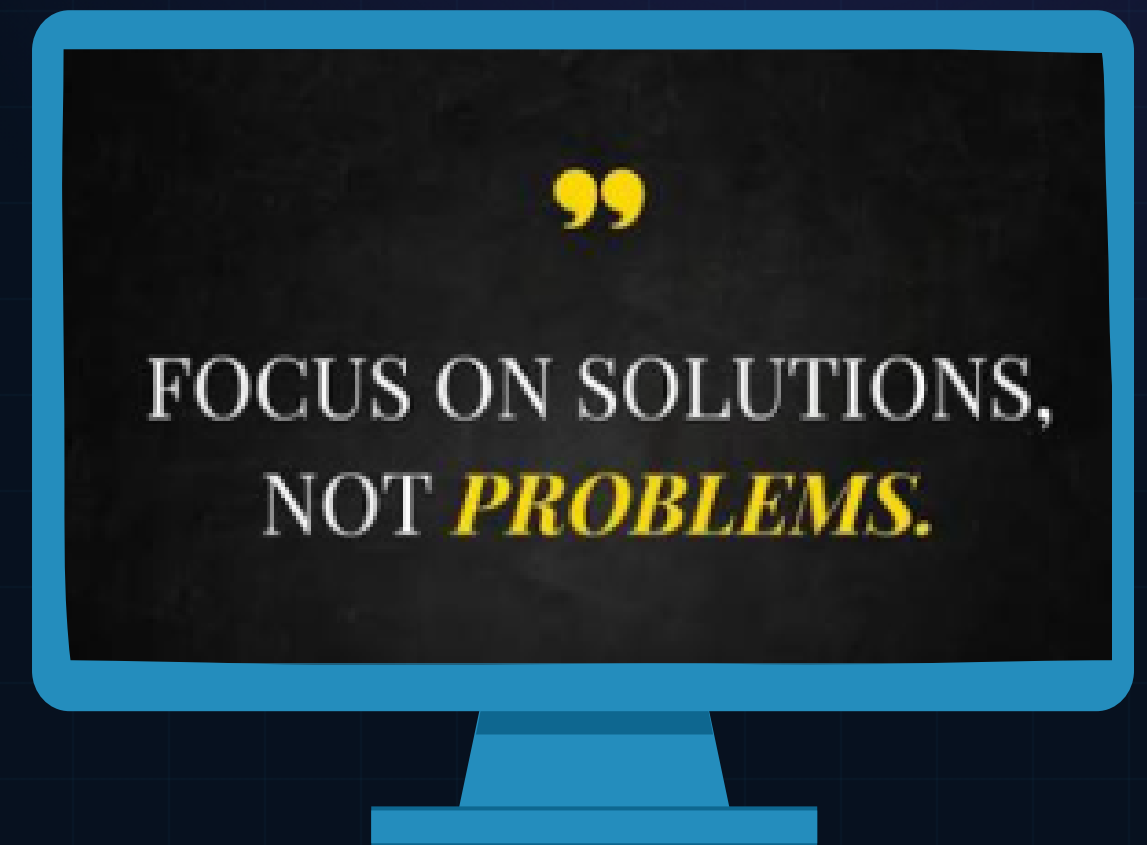
Distributing all these operations across various node classes leads to a system that's hard to understand, maintain, and change.

Adding a **NEW** operation would require to recompile **ALL** of these classes!



We Need a Solution Such That:

- 01 Each new operation can be added separately.
- 02 The node classes are independent of the operations that apply to them. (Hence not polluted!)



Introduction

VISITOR IS A BEHAVIORAL PATTERN.

Represent an operation to be performed on the elements of an object structure.

Visitor lets you define a new operation without changing the classes of the elements on which it operates.

Introduction

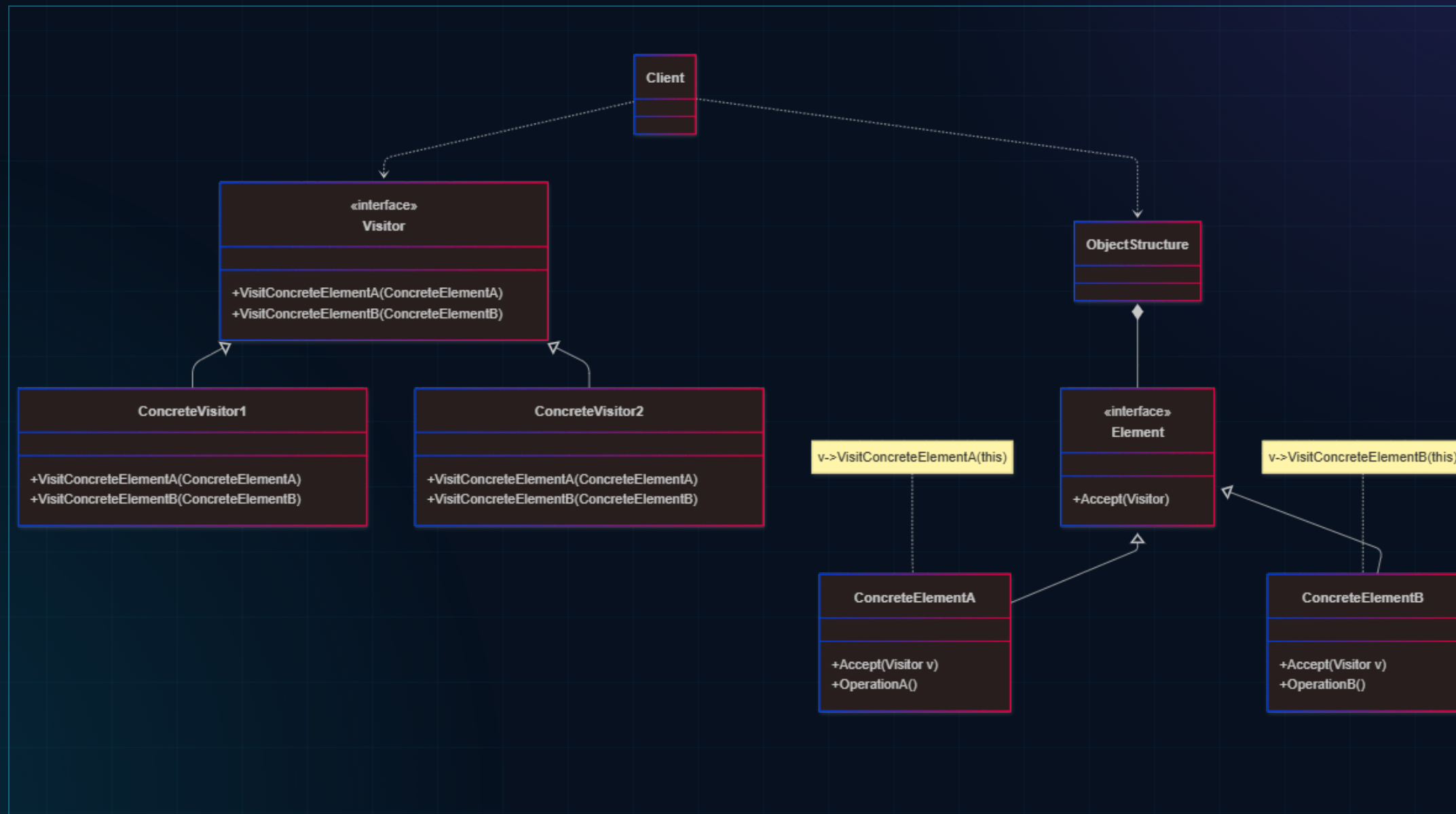
We can package related operations from each class in a separate object, called a visitor, and passing it to elements of the abstract syntax tree as it's traversed.

When an element "accepts" the visitor, it sends a request to the visitor that encodes the element's class. It also includes the element as an argument. The visitor will then execute the operation for that element -- the operation that used to be in the class of the element.

Applicability (When to Use)

- Have an object structure with many different classes that have differing interfaces.
- Need to perform distinct, unrelated operations without “polluting” your clean data classes.
- **NOTE:** If your data classes change frequently, this pattern is a bad fit because changing the object structure requires redefining the interface for all visitors.

General Diagram



The Flow

01**CREATING**

The client creates a Visitor

**02****TRAVERSING**

Walks through the object structure

**03****CALLING**

Calls Accept(Visitor) on each element

**04****CALLING**

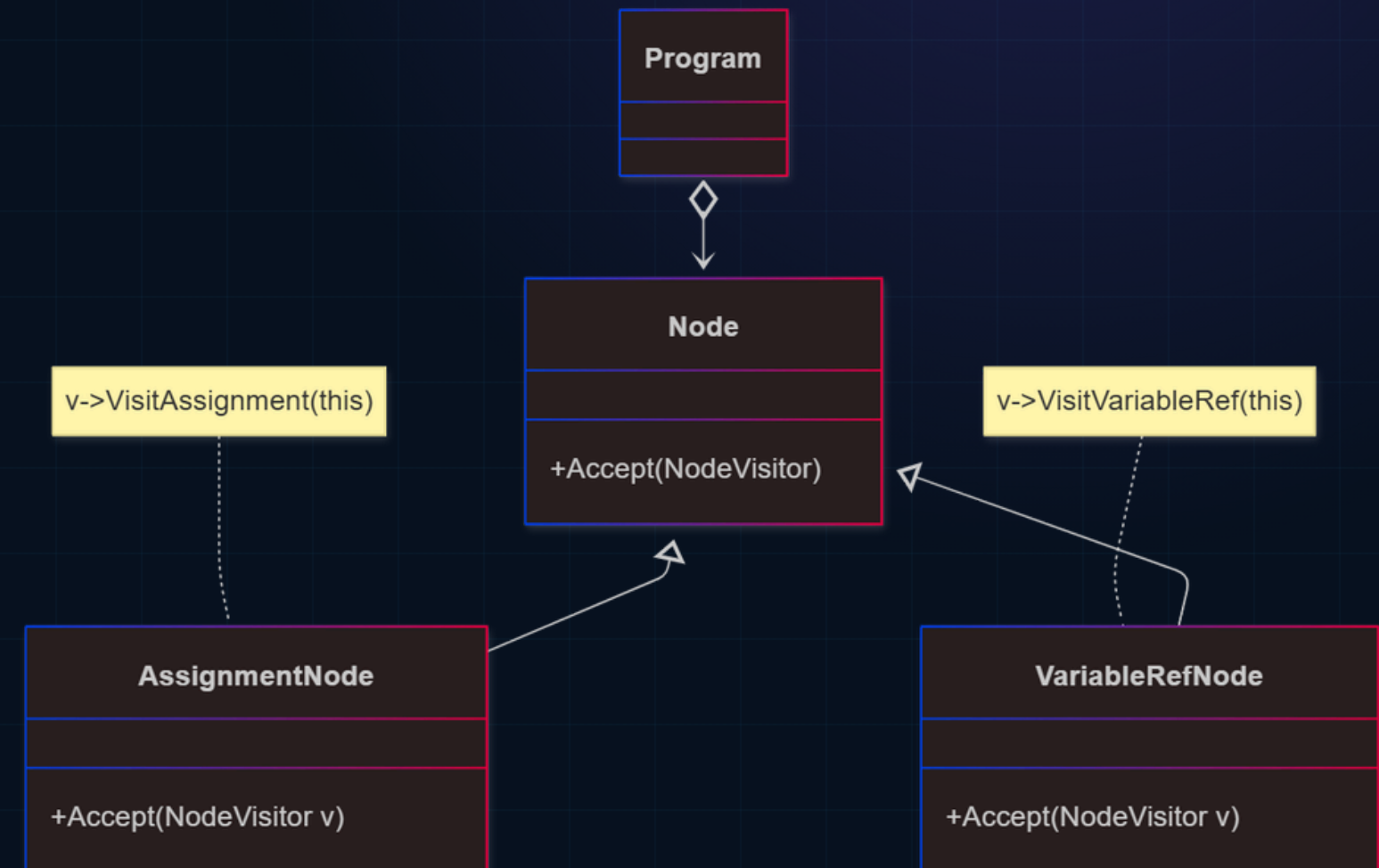
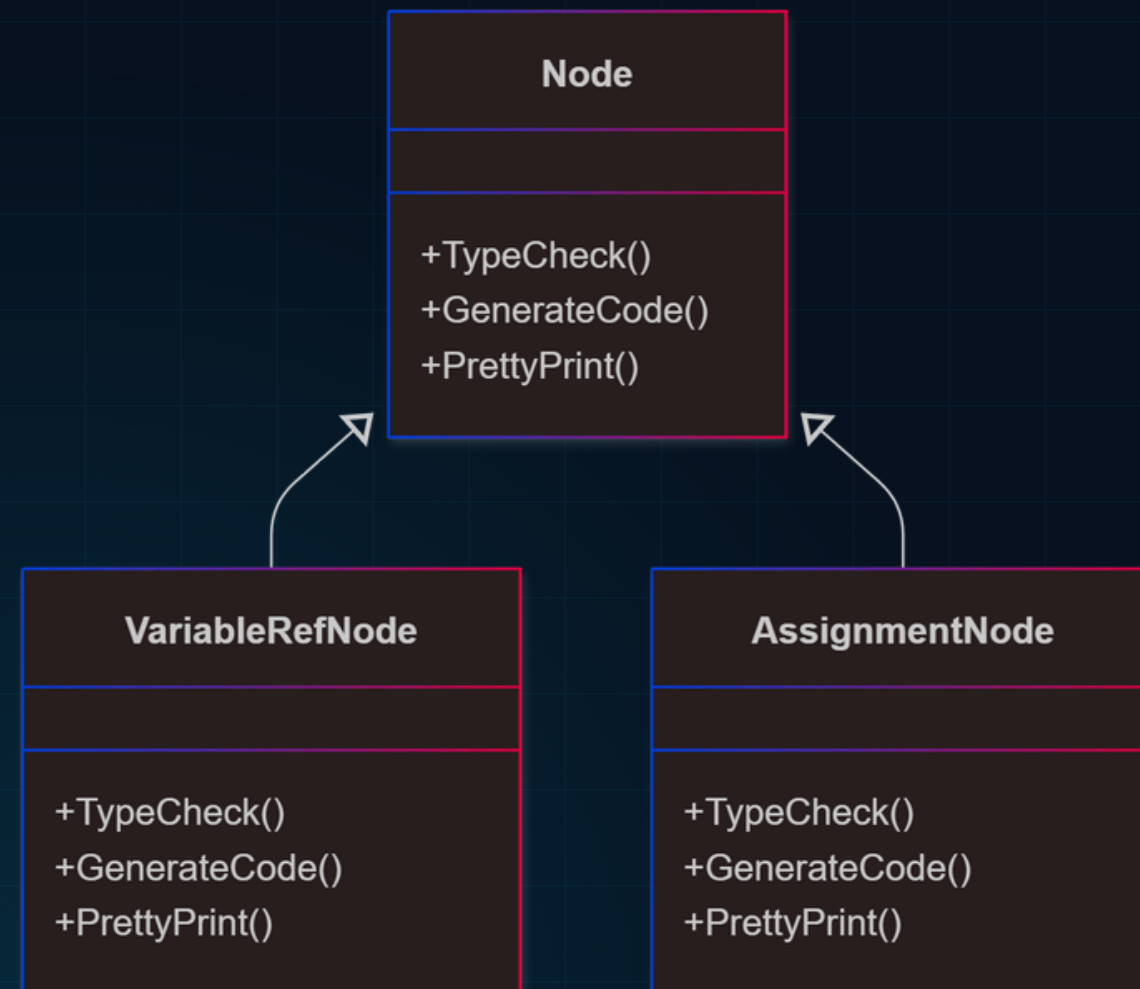
The element calls the corresponding Visit operation of the visitor, passing itself as an arg.

Coming Back to the Compiler Problem

NAIVE APPROACH

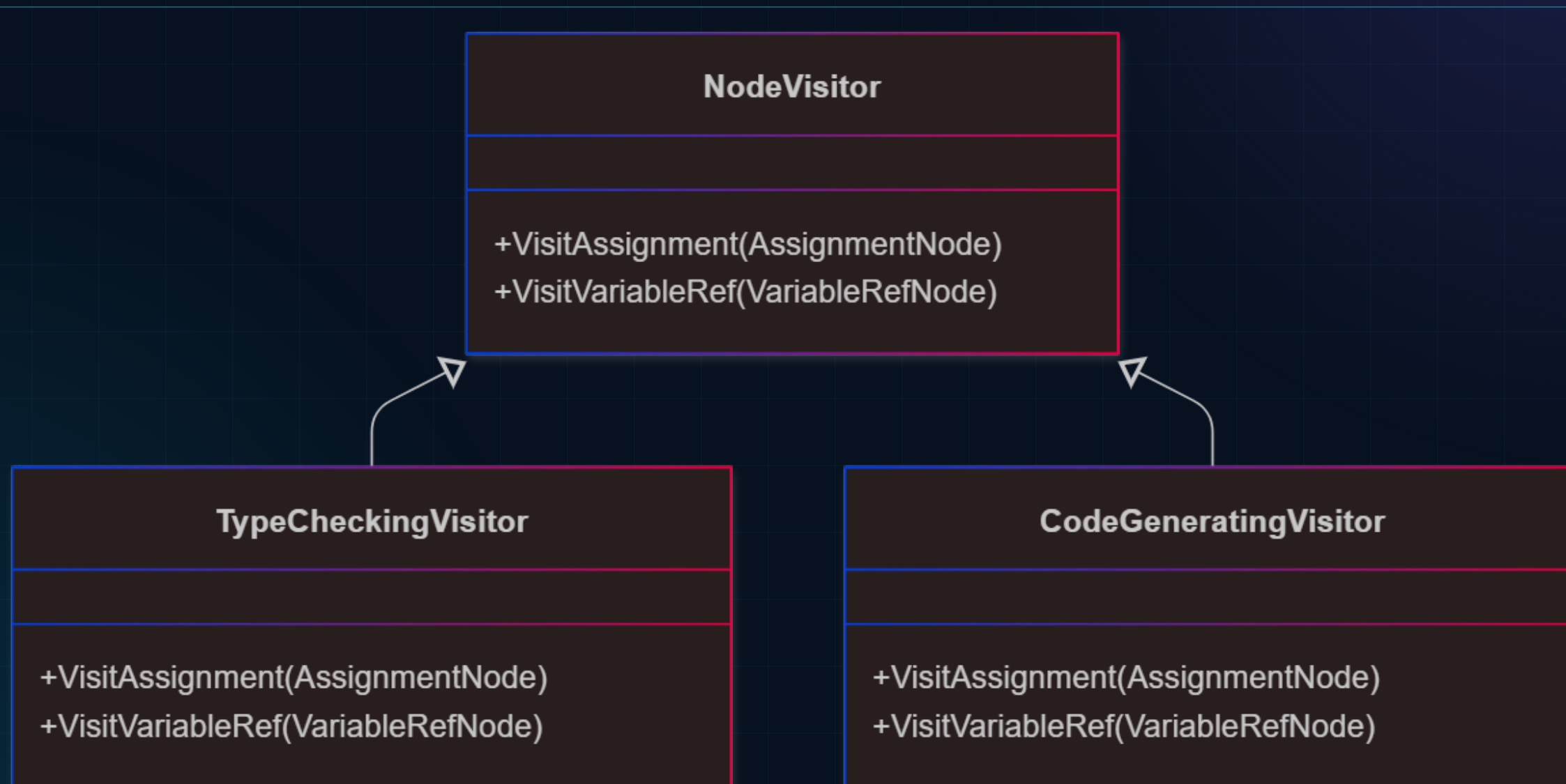
VS

OPTIMIZED APPROACH (WITH VISITOR)



Coming Back to the Compiler Problem

OPTIMIZED APPROACH (WITH VISITOR)



The Blueprint

Visitor:

An interface declaring a Visit operation for every specific concrete element in your system.



```
1 // The Visitor Interface
2 class Visitor {
3     public:
4         virtual ~Visitor() = default;
5         virtual void VisitElementA(ElementA*) = 0;
6         virtual void VisitElementB(ElementB*) = 0;
7         //and so on for other concrete elements
8     protected:
9         Visitor();
10 };
```

The Blueprint



```
1 // Concrete Visitor
2 class ConcreteVisitor: public Visitor {
3 public:
4     ConcreteVisitor();
5     void VisitElementA(ElementA*) override { /* Logic for A*/}
6     void VisitElementB(ElementB*) override { /* Logic for B*/}
7     //and so on for other concrete elements
8 };
```

ConcreteVisitor:

The actual tool doing the work which implements the operations and stores any local state it needs.

The Blueprint

Abstract Element:

Your data classes (like nodes) that must define an Accept(Visitor) operation.



```
1 // Abstract Element
2 class Element {
3 public:
4     virtual ~Element() = default;
5     virtual void Accept(Visitor&) = 0;
6 protected:
7     Element();
8 };
```

Concrete Element:

Defines the Accept(Visitor) operation by passing itself as an argument for the visitElementA function.



```
1 // Concrete Element
2 class ElementA: public Element{
3 public:
4     ElementA();
5     void Accept(Visitor& v) override{
6         // Double-dispatch
7         v.VisitElementA(this);
8     }
9 };
```

The Blueprint

Object Structure:

- Can enumerate its elements.
- May provide a high-level interface to allow visitors to visit its elements.
- May either be a **Composite** or a **collection** such as a list/set/vector etc.

```
1 // Composite Object Structure (Inherits from elements
2 class CompositeElement: public Element {
3 private:
4     vector<Element*> children;
5 public:
6     void AddChild(Element* child) {
7         children.push_back(child);
8     }
9     void Accept(Visitor& v) override {
10         for (auto child : children) {
11             child->Accept(v);
12         }
13         v.VisitCompositeElement(this);
14     }
```


Double-dispatch Magic

Single-dispatch (e.g. C++): a method is executed based on the name of the request and the type of receiver.

e.g. As for the naive approach in the Compiler Problem, calling `node->GenerateCode()` :

```
node->GenerateCode()
```



The compiler dynamically check the runtime type of node.



If it is `VariableRefNode`, then :
`VariableRefNode::GenerateCode()`

Double-dispatch Magic

Visitor Pattern relies on *Double-dispatch*, meaning the executed operation depends on the **name of the request** and **2 types of receiver**: the specific type of Visitor and Element.

Since C++ does not support this technique, Visitor “fakes” it by applying single-dispatch twice in a row.

e.g. The accept() method:

Dispatch 1(The element):
calls `element->Accept(visitor)`



The compiler dynamically check the runtime type of the element.
Calls `VariableRefNode::Accept()`



Dispatch 2(The Visitor): calls
`Visitor->visitVariableRefNode(this)`



The compiler dynamically check the runtime type of the visitor.
Calls `TypeCheckingVisitor->visitVariableRefNode(this)`

Who Is Responsible for Traversal?

A visitor must visit each element of the object structure. Question is, how does it get there?

01

The Object Structure (Most Common)

The responsibility often lies within the object structure itself. A collection can simply iterate over its list calling `Accept`, or a `Composite` can traverse itself by recursively calling `Accept` on its children.

02

A Separate Iterator

You can use a dedicated `Iterator` object (internal or external) to walk through the data structure and pass the visitor to each element.

03

The Visitor Itself (Use with Caution)

You can put the traversal algorithm directly inside the visitor. However, this often duplicates traversal code across every `Concrete Visitor`. The main reason to do this is if you have a highly complex traversal that changes based on the results of the operations.

Sample Code

Problem:

Create a simple shopping cart system that have different items (Books and Fruits), and calculate the total price without stuffing pricing logic directly into the item classes.

- Books over \$50 get a \$5 discount
- Fruits are priced by weight

Sample Code

```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  // Forward declaration to avoid circular dependencies
5  class Book;
6  class Fruit;
7  // Visitor interface
8  class ShoppingCartVisitor {
9  public:
10     virtual ~ShoppingCartVisitor() = default;
11     virtual void visit(Book* book) = 0;
12     virtual void visit(Fruit* fruit) = 0;
13 protected:
14     ShoppingCartVisitor() = default;
15 };
16 // Element interface of the Object Structure
17 class ItemElement {
18 public:
19     virtual ~ItemElement() = default;
20     virtual void accept(ShoppingCartVisitor& visitor) = 0;
21 protected:
22     ItemElement() = default;
```

Sample Code



```
1 // Concrete elements
2 class Book : public ItemElement {
3 private:
4     double price;
5     std::string title;
6 public:
7     Book(double price, std::string title) : price(price), title(title) {}
8     double getPrice() const { return price; }
9     std::string getTitle() const { return title; }
10    void accept(ShoppingCartVisitor& visitor) override {
11        visitor.visit(this);
12    }
13 };
14
15 class Fruit : public ItemElement {
16 private:
17     double weight;
18     double pricePerKg;
19     std::string name;
20 public:
21     Fruit(double weight, double pricePerKg, std::string name)
22         : weight(weight), pricePerKg(pricePerKg), name(name) {}
23     double getWeight() const { return weight; }
24     double getPricePerKg() const { return pricePerKg; }
25     std::string getName() const { return name; }
26     void accept(ShoppingCartVisitor& visitor) override {
27         visitor.visit(this);
28     }
29 }
```

Sample Code

```
1 // The concrete Visitor
2 class CalculatorVisitor: public ShoppingCartVisitor{
3 private:
4     double totalCost = 0.0;
5 public:
6     void visit(Book* book) override {
7         // Books over $50 get a $5 discount
8         double cost = book->getPrice();
9         if (cost > 50.0) { cost -= 5.0; }
10        std::cout << "Book (" << book->getTitle() << ") Cost: $" << cost << "\n";
11        totalCost += cost;
12    }
13    void visit(Fruit* fruit) override {
14        // Fruits are priced by weight
15        double cost = fruit->getWeight() * fruit->getPricePerKg();
16        std::cout << "Fruit (" << fruit->getName() << ") Cost: $" << cost << "\n";
17        totalCost += cost;
18    }
19    double getTotalCost() const {
20        return totalCost;
21    }
22 };
```

Sample Code



```
1 // The client (main method)
2 int main() {
3     Book book1(20.0, "Design Patterns");
4     Book book2(60.0, "Advanced Algorithms");
5     Fruit fruit1(2.5, 4.0, "Apples");
6     std::vector<ItemElement*> items = { &book1, &book2, &fruit1 };
7     CalculatorVisitor visitor;
8     for (ItemElement* item : items) {
9         item->accept(visitor);
10    }
11    std::cout << "\nTotal Cart Cost: $" << visitor.getTotalCost() << "\n";
12    return 0;
13 }
```


Advantages

01

“Easily” adding new operators for existing classes

02

Visitors can work across objects that don't even share a common parent class.

03

Visitors can accumulate state as they travel through the elements, replacing global variables.

Disadvantages

01

Adding new concreteElement is really painful

03

It often forces you to expose an element's internal state through public operations so the visitor can do its job, which can break data encapsulation.

02

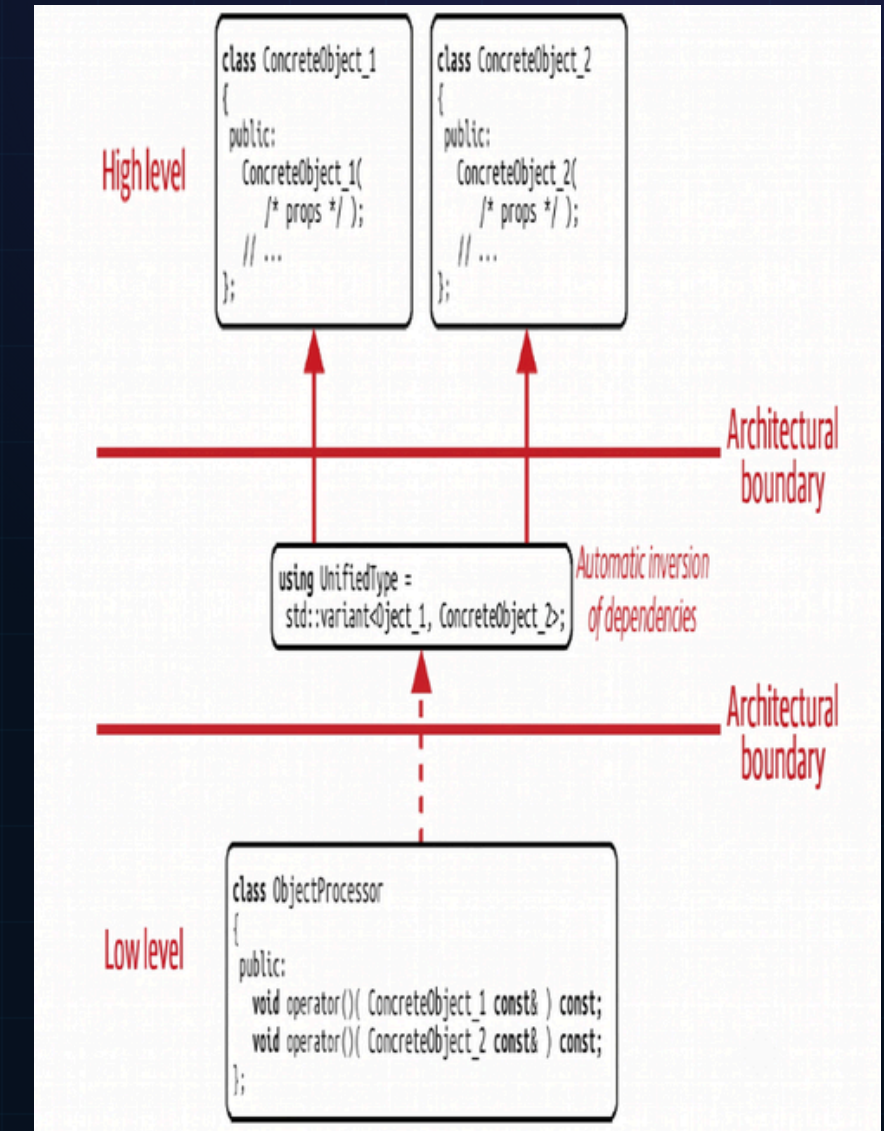
Cyclic dependencies make you harder to change/ affect the structure

04

Intrusive nature- you have to implement accept() for “visited” classes

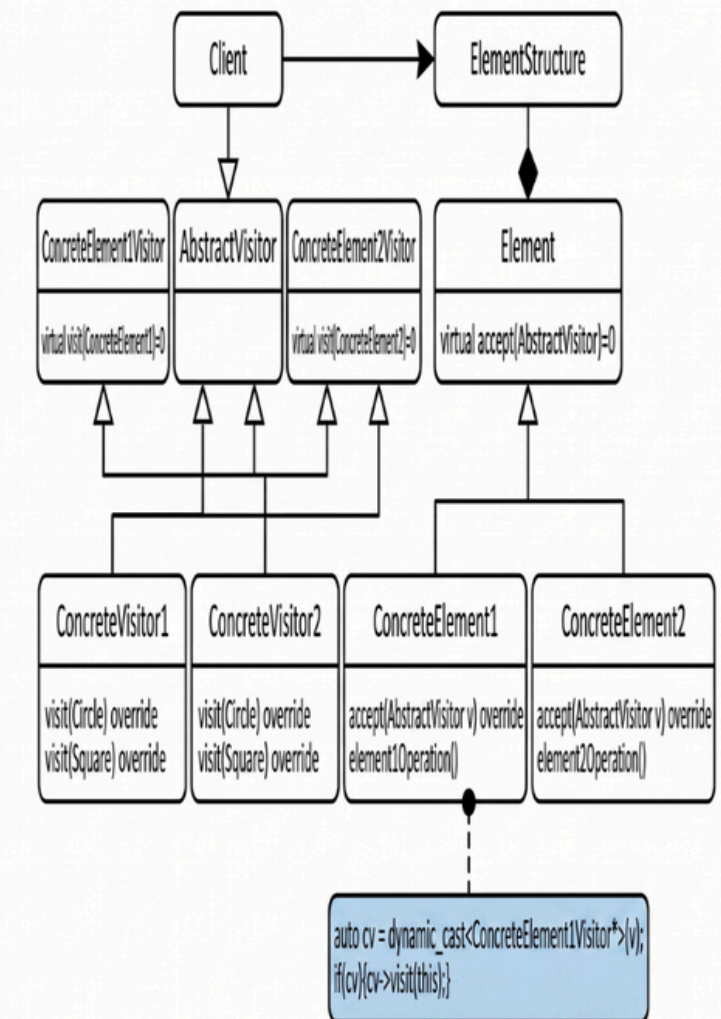
Variations- std::variant

- Adopts a procedural programming paradigm while operating similarly to the Classic Visitor pattern, leading to widespread adoption.
- Exhibits a non-intrusive design regarding the visited classes.
- Minimizes base-class dependencies, granting operators enhanced flexibility over the specific classes they process.
- Transitions from dynamic to static polymorphism due to yields substantial performance improvements.
- Limitation: Retains the fundamental drawback of the classic Visitor pattern—the inherent difficulty in incorporating new data types.



Variations- Acyclic Visitor

- Utilizes `dynamic_cast` to perform cross-casting.
- Facilitates the seamless addition of new Data Types without impacting existing Visitors.
- Allows the introduction of new Visitors without modifying the Element classes.
- Enables Visitors to selectively target specific ConcreteElements, significantly reducing inter-component coupling.
- Limitation: Remains an intrusive design, as it necessitates the implementation of the `accept()` method within the Elements.
- Limitation: The reliance on `dynamic_cast` for cross-casting incurs severe performance overhead.



Application in Web Dev

JavaScript Compilers (Babel & TypeScript)

The Problem: Browsers can't read modern TypeScript or JSX (React) directly. It has to be translated into plain, older JavaScript.

The Visitor Solution: Tools like Babel and the TypeScript compiler parse your code into a massive Abstract Syntax Tree (AST). They then use the Visitor pattern to walk through that tree. When the visitor hits an ArrowFunctionNode or an InterfaceNode, it executes the logic to transform it into older JavaScript.

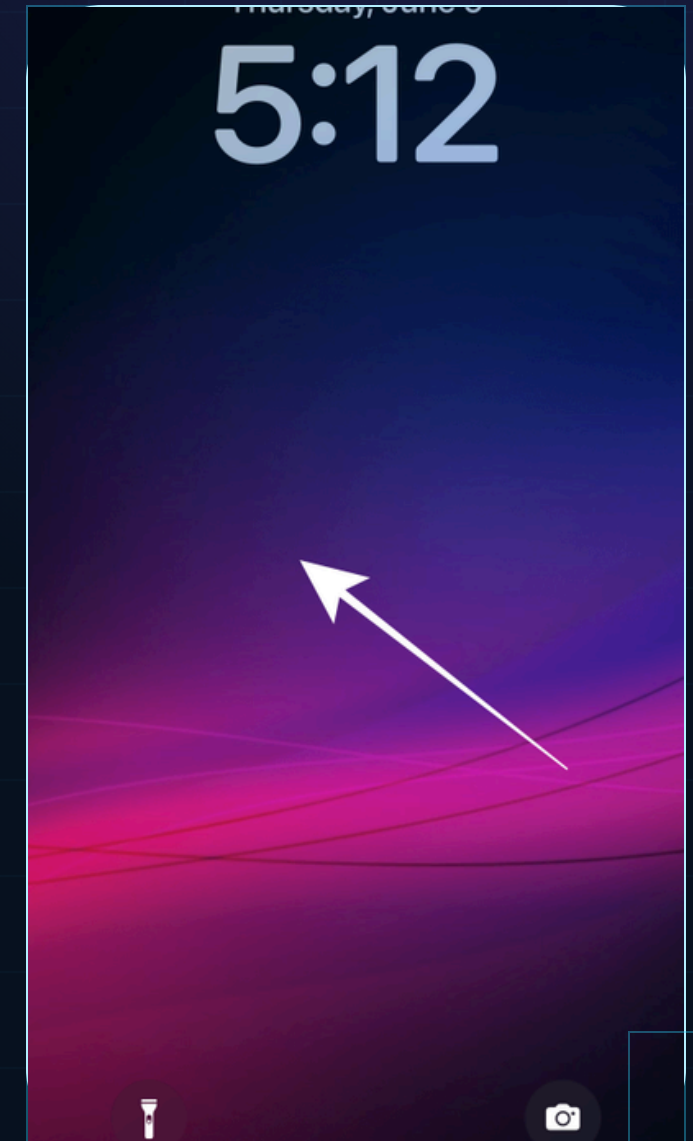


Application in Mobile Dev

UI View hierarchy traversal:

The Problem: Every screen on a mobile app is just a giant Composite tree of UI elements (a Screen contains a ScrollView, which contains Buttons, TextViews, and Images). How to traverse through all of these to change theme?

The Visitor Solution: You implement a ThemeVisitor. You pass it to the root of the screen, and it recursively visits every child element. When it visits a TextView, it changes the font color to white. When it visits a View background, it changes it to dark gray. The UI elements just accept the visitor without caring about the theme logic.



Comparison Between 3 Patterns

Pattern	Core Mission	Routing Strategy	Best Used When...
Iterator	Traversal	Sequential: Moves from object A to object B mechanically.	You need to walk through a collection without exposing its underlying structure (like arrays vs. trees).
Chain of Responsibility	Decoupling Senders & Receivers	"Pass the Buck": Passes a request down a line until someone decides to handle it.	The exact object that needs to process a request isn't known at compile-time.
Visitor	Operation Encapsulation	"Precision Strike": Uses double-dispatch to instantly bind the exact operation to the exact element.	The data structure is completely stable, but you frequently add new operations to it.

Relations of the Patterns

The same style

All three belong to the Behavioral Pattern family, meaning their main goal is managing

control flow and communication

between objects rather than just creating them (Creational) or structuring them (Structural).

Relations of the Patterns

Chain of responsibility + Visitor:
The “Who vs What” dynamic

WHO

Chain of Responsibility focuses on who will handle a request (routing it dynamically until a handler accepts it).



WHAT

Visitor focuses on what to do with a request (applying strict, pre-defined logic to a known element type).

Relations of the Patterns

Iterator + Visitor: perfect combination

TRAVEL

The Iterator is responsible for the travel (walking through a complex object structure).

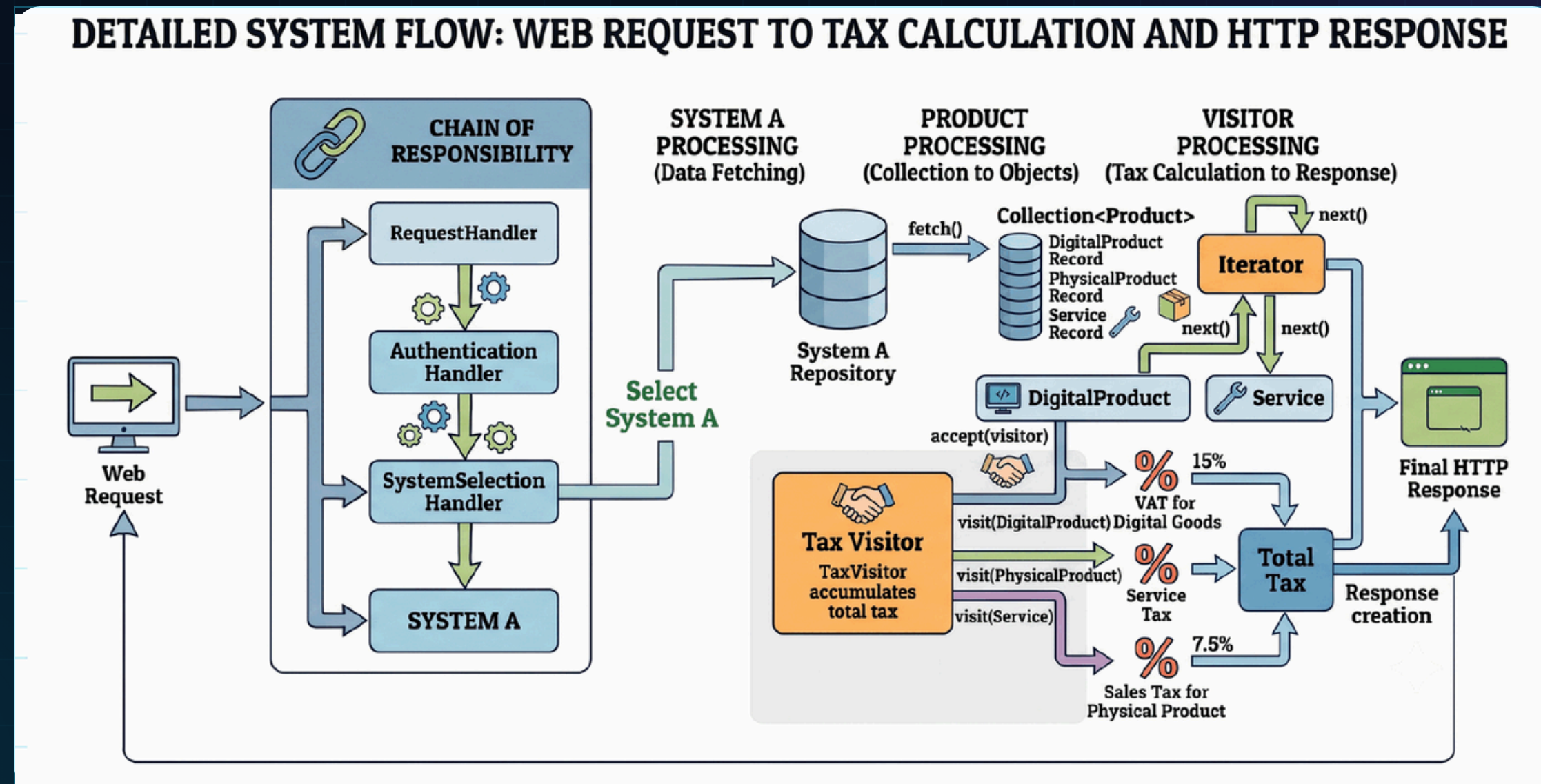
+

WORK

The Visitor is responsible for the work (executing specific logic at every stop).

Relations of the Patterns

An ultimate combination



Quiz

Q1: What type of dispatch is naturally applied by C++, and what type of dispatch is simulated by the Visitor pattern?

A. Double-dispatch, single-dispatch

B. Both single-dispatch

C. Both double-dispatch

D. Single-dispatch, double-dispatch

Quiz

Q1: What type of dispatch is naturally applied by C++, and what type of dispatch is simulated by the Visitor pattern?

A. Double-dispatch, single-dispatch

B. Both single-dispatch

C. Both double-dispatch

D. Single-dispatch, double-dispatch

Quiz

Q2: When is the BEST time to use the Visitor pattern?

- A.** When the classes defining the object structure change frequently.
- B.** When the object structure is completely stable, but you often need to define new operations over it.
- C.** When you need to hide the creation logic of an object.
- D.** When you want to pass a request along a chain of objects until it is handled.

Quiz

Q2: When is the BEST time to use the Visitor pattern?

- A.** When the classes defining the object structure change frequently.
- B.** When the object structure is completely stable, but you often need to define new operations over it.
- C.** When you need to hide the creation logic of an object.
- D.** When you want to pass a request along a chain of objects until it is handled.

Quiz

Q3: What is one of the main negative consequences (drawbacks) of using the Visitor pattern?

A. It makes adding new operations incredibly difficult.

B. It can break encapsulation by forcing elements to expose their internal state via public methods.

C. It prevents visitors from accumulating state during traversal.

D. It forces all elements to share a single, strict parent class.

Quiz

Q3: What is one of the main negative consequences (drawbacks) of using the Visitor pattern?

A. It makes adding new operations incredibly difficult.

B. It can break encapsulation by forcing elements to expose their internal state via public methods.

C. It prevents visitors from accumulating state during traversal.

D. It forces all elements to share a single, strict parent class.

Quiz

Q4: The "Object Structure" in the Visitor pattern is responsible for enumerating its elements. What other design pattern is most commonly used to implement this structure?

A. The Composite Pattern

B. The Singleton Pattern

C. The Prototype Pattern

D. The Factory Method Pattern

Quiz

Q4: The "Object Structure" in the Visitor pattern is responsible for enumerating its elements. What other design pattern is most commonly used to implement this structure?

A. The Composite Pattern

B. The Singleton Pattern

C. The Prototype Pattern

D. The Factory Method Pattern

Quiz

Q5: Where can the responsibility for traversing (iterating through) the object structure be placed?

- A.** It must always be hardcoded directly into the ConcreteVisitor class.
- B.** It must always be handled by the base Element class.
- C.** It can be placed in the Object Structure, a separate Iterator, or even in the Visitor itself.
- D.** It is automatically handled by the C++ compiler during double-dispatch.

Quiz

Q5: Where can the responsibility for traversing (iterating through) the object structure be placed?

- A.** It must always be hardcoded directly into the ConcreteVisitor class.
- B.** It must always be handled by the base Element class.
- C.** It can be placed in the Object Structure, a separate Iterator, or even in the Visitor itself.
- D.** It is automatically handled by the C++ compiler during double-dispatch.

```
visitor.complete();
```

THANKS!

Do you have any questions?